

# 作业感受

姜佳岩 · 第一关挑战 · 2026-09-27

## 一、我如何完成第一次挑战

我先在 Windows 上启用 WSL，再到 Ubuntu 中建立独立的 Python 环境，安装 SGLang 0.5.14 和 Ray 2.56.0，用本地 RTX 4060 运行 Qwen/Qwen3-0.6B。启动服务后，我通过 `/v1/models` 核对模型名称，再发一条聊天请求，确认推理正常。之后，我按长度筛选 Mooncake 历史 trace 中的记录，用固定随机种子抽取 20 条，合成提示词，按泊松到达时间发送请求，记录 token 数、HTTP 状态和总耗时。

操作时，我先检查 GPU、Python 和 CUDA，再检查模型接口，最后试了一条长输入请求。确认单条请求正常后，才发送全部 20 条，结果都返回了 HTTP 200。这样一步步检查会慢一些，但出了问题更容易找到原因。

## 二、最困难的部分与解决过程

环境兼容问题花的时间最多。最初使用 Ubuntu 26.04 和 CUDA Toolkit 13.0 时出现了编译错误。换成 Ubuntu 24.04 后，又陆续遇到缺少 C 编译器、nvcc、Python 3.12 开发头文件，以及 CUTLASS DSL 依赖冲突的问题。我照着日志逐项处理，每改完一项就重新启动服务，直到实际请求也能成功返回。

脚本也出过问题。第一次统计 token 时，我直接对分词器的返回对象使用 `len()`，得到的结果是 2，后来才发现数到的是返回字段。改为统计 `input_ids` 后，长提示词的输入量是 3073 token，与服务返回的 `usage` 一致。这次出错提醒了我：程序没有报错，也不代表结果就对，还是得核对。

## 三、对从 0 到 1 研究工作的启发

这次只测试了 20 条请求，平均到达率是 0.5 次/秒。输入由重复文字组成，没有按 trace 的 `hash_ids` 重建共享前缀，也没有测 TTFT，或在同一台机器上用 vLLM 做对照。所以，这些数据只能说明本机服务已经跑通，还不能用来判断哪种缓存机制更快。以后研究性能时，我需要先确定要比较的指标，保留原始数据和脚本，再用更接近真实请求的输入和对照组来检验结论。

仓库：<https://github.com/Malone208/First-homework>